

CHANDIGARH UNIVERSITY APEX INSTITUTE OF TECHNOLOGY



Assignment 2

Subject: Computer Networks

Submitted By:
Name: Vishesh Kansugia
UID: 24BAI70184
Class: 24AIT_KRG_1B

Submitted To:
Mr. Alok Sir

Q1. What is an IP Address? Why do we need classes in IPv4? Explain each class, mentioning the number of hosts and networks possible. Also explain subnetting and supernetting with examples.

1.1 IP Address

An Internet Protocol (IP) address is a logical address assigned to a device connected to a network. It helps identify the source and destination of data packets and enables communication between devices over the internet or a local network.

An IPv4 address is 32 bits long and is divided into four octets, each containing 8 bits. It is generally represented in dotted-decimal notation, such as 192.168.1.10.

An IP address consists of two main parts: the Network ID and the Host ID. The Network ID identifies the network to which a device belongs, while the Host ID identifies the particular device within that network.

1.2 Need for IPv4 Address Classes

IPv4 provides a total of 2^{32} , or approximately 4.3 billion, possible addresses. However, not every organization requires the same number of IP addresses. A large organization may need thousands of addresses, whereas a small office may need only a few hundred.

To accommodate networks of different sizes, IPv4 addresses were originally divided into five classes: A, B, C, D and E. This system is known as classful addressing.

Classes A, B and C were designed for networks of different sizes. Class D is used for multicasting, while Class E was reserved for experimental purposes.

1.3 IPv4 Address Classes

Class A

Class A addresses were designed for very large networks. The first octet identifies the network, while the remaining three octets identify hosts.

The first bit is fixed as 0, leaving 7 bits for network identification and 24 bits for hosts.

- Address range: 1.0.0.0 to 126.255.255.255
- Default subnet mask: 255.0.0.0 (/8)
- Number of usable networks: $2^7 - 2 = 126$
- Usable hosts per network: $2^{24} - 2 = 16,777,214$

For example, in the address 10.20.30.40, the first octet, 10, represents the network, while the remaining octets identify the host.

Class B

Class B addresses were intended for medium-sized networks. The first two octets represent the network, and the last two represent the host.

The first two bits are fixed as 10, leaving 14 bits for network identification and 16 bits for hosts.

- Address range: 128.0.0.0 to 191.255.255.255
- Default subnet mask: 255.255.0.0 (/16)
- Number of networks: $2^{14}=16,384$
- Usable hosts per network: $2^{16}-2=65,534$

For example, in 172.16.20.30, the network portion is 172.16, while 20.30 represents the host portion.

Class C

Class C addresses were designed for smaller networks. The first three octets identify the network, while the final octet identifies the host.

The first three bits are fixed as 110, leaving 21 bits for network identification and 8 bits for hosts.

- Address range: 192.0.0.0 to 223.255.255.255
- Default subnet mask: 255.255.255.0 (/24)
- Number of networks: $2^{21}=2,097,152$
- Usable hosts per network: $2^8-2=254$

For example, in 192.168.1.20, the network portion is 192.168.1, and 20 identifies the host.

Class D

Class D addresses range from 224.0.0.0 to 239.255.255.255 and are used for multicasting. Multicasting allows data to be transmitted to a group of devices simultaneously. These addresses do not have the traditional network and host division.

Class E

Class E addresses range from 240.0.0.0 to 255.255.255.255. They were reserved primarily for experimental purposes and are not used for ordinary host addressing.

Summary of IPv4 Classes

Class	First Octet	Default Mask	Networks	Hosts per Network
A	1–126	/8	126	16,777,214
B	128–191	/16	16,384	65,534
C	192–223	/24	2,097,152	254
D	224–239	—	Multicast	—
E	240–255	—	Experimental	—

Today, classful addressing has largely been replaced by Classless Inter-Domain Routing (CIDR), which provides more flexible allocation of IP addresses.

1.4 Subnetting

Subnetting is the process of dividing a large network into smaller logical networks called subnets. It is achieved by borrowing bits from the host portion of an IP address and using them as additional network bits.

Subnetting is useful because it allows an organization to divide its network into separate sections, such as different departments. It also improves address utilization, reduces broadcast traffic and makes network management easier.

Example: Consider the network 192.168.1.0/24. Suppose an organization wants to divide it into four equal subnets.

To create four subnets, two bits must be borrowed from the host portion because:

$$2^2 = 2^2 = 4$$

The new prefix becomes /26, and the subnet mask becomes 255.255.255.192.

	Subnet	Network Address	Usable Host Range
1	192.168.1.0/26	192.168.1.1–62	
2	192.168.1.64/26	192.168.1.65–126	
3	192.168.1.128/26	192.168.1.129–190	
4	192.168.1.192/26	192.168.1.193–254	

Each subnet has 6 host bits, allowing $2^6 - 2$ usable host addresses.

1.5 Supernetting

Supernetting is the process of combining multiple smaller, contiguous networks into one larger network. It is commonly used for route aggregation, which reduces the number of entries required in routing tables.

For example, consider four networks: 192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24 and 192.168.3.0/24.

These networks can be combined into a single supernet, 192.168.0.0/22. The resulting network covers the address range from 192.168.0.0 to 192.168.3.255.

Thus, subnetting divides a network into smaller networks, whereas supernetting combines smaller networks into a larger address block.

Q2. What is the difference between Link State and Distance Vector Routing? Explain how these algorithms identify the least-cost path with examples.

2.1 Introduction to Routing

Routing is the process of selecting a path through which data packets travel from a source to a destination. Since multiple paths may exist between two devices, routers use routing algorithms to determine the most suitable path based on metrics such as hop count, delay, bandwidth or link cost.

Two important routing approaches are Distance Vector Routing and Link State Routing. Both aim to determine efficient routes, but they differ in how routers collect information and calculate paths.

2.2 Distance Vector Routing

In Distance Vector Routing, each router maintains a routing table containing the estimated cost and next hop for reaching different destinations. Routers exchange routing information with their directly connected neighbors rather than maintaining a complete map of the network.

This approach is based on the Bellman-Ford algorithm. Each router calculates the cost of reaching a destination through its neighbors and selects the path with the minimum total cost.

Example

Consider a network with four routers, A, B, C and D.

Suppose router A wants to reach router B. The direct path A → B has a cost of 2, while the alternative path A → C → D → B has a total cost of:

$$1+1+3=5$$

Router A compares the available costs learned through its neighbours and selects the direct path A → B because its cost is lower.

Distance Vector Routing is relatively simple to implement. However, it may take longer to converge after a network failure and can suffer from routing loops and the count-to-infinity problem.

2.3 Link State Routing

In Link State Routing, every router develops a complete view of the network topology. Instead of exchanging entire routing tables with neighbours, routers distribute information about their directly connected links and their costs.

Each router discovers its neighbours, measures link costs and creates link-state advertisements. These advertisements are flooded throughout the routing area so that routers can build a consistent topology database.

Once the topology is known, each router uses Dijkstra's Shortest Path First algorithm to calculate the least-cost paths from itself to other routers.

Example

Router A has information about all four routers and the costs of their links.

Dijkstra's algorithm begins at router A with a cost of zero. It then examines its directly connected neighbors: B with cost 2 and C with cost 1. Router C is selected first because it has the lowest tentative cost. Through C, router D can be reached at a total cost of 2.

The algorithm then evaluates the remaining paths. Router B can be reached directly from A at cost 2, while the path through C and D has cost 5.

Therefore, the least-cost path from A to B is A → B, with a total cost of 2.

Link State Routing generally converges faster after topology changes, but it requires more memory and processing resources to maintain the topology database and calculate shortest paths.

2.4 Difference Between Distance Vector and Link State Routing

Basis	Leaky Bucket	Token Bucket
Working principle	Packets are transmitted at a fixed rate	Tokens permit packet transmission
Burst traffic	Smooths incoming bursts	Allows controlled bursts
Bucket stores	Packets or data	Tokens
Unused capacity	Does not accumulate transmission credits	Unused tokens accumulate
Excess traffic	Packets may be discarded if the queue fills	Packets must wait or may be dropped if tokens are insufficient
Main purpose	Produces smooth traffic	Controls average rate while allowing bursts

Both approaches determine least-cost routes, but Distance Vector relies on information exchanged between neighboring routers, while Link State calculates paths using a broader view of the network.

Q3. What is congestion in a network? Why do we need congestion control? Explain the difference between Leaky Bucket and Token Bucket with examples. Also explain why fragmentation is needed.

3.1 Network Congestion

Network congestion occurs when the amount of traffic entering a network exceeds the capacity of its links, routers or other networking devices to handle that traffic efficiently.

Every router has limited processing capacity and buffer space. When packets arrive faster than they can be forwarded, they begin accumulating in queues. If the buffers become full, additional packets may be dropped.

For example, suppose a router can forward 100 packets per second but receives 150 packets per second. The excess packets must wait in the queue. If this situation continues, the queue may fill up, resulting in packet loss and increased delay.

3.2 Need for Congestion Control

Congestion control is necessary to maintain reliable and efficient network communication. Without it, excessive traffic can cause long delays, packet loss and repeated retransmissions.

Retransmissions can further increase network traffic, making congestion worse. In severe cases, the network may spend a significant amount of its capacity handling retransmitted packets rather than delivering new data.

Congestion-control mechanisms help regulate the amount of traffic entering the network, reduce packet loss and maintain better throughput. Leaky Bucket and Token Bucket are two commonly studied traffic-shaping mechanisms that can help control traffic entering a network.

3.3 Leaky Bucket Algorithm

The Leaky Bucket algorithm regulates outgoing traffic by transmitting packets at a fixed rate. It can be compared to a bucket with a small hole at the bottom: water may enter at different rates, but it leaves at a steady rate.

Similarly, packets may arrive in bursts, but they are placed in a queue and transmitted at a controlled rate. If the queue is full, newly arriving packets may be discarded.

Example: Suppose a bucket can store five packets and has an output rate of two packets per second. If five packets arrive simultaneously, two packets are transmitted during the first second, two during the second second and the remaining packet during the third second.

The algorithm therefore converts irregular incoming traffic into smoother outgoing traffic.

3.4 Token Bucket Algorithm

The Token Bucket algorithm allows controlled bursts of traffic while limiting the average transmission rate.

In this method, tokens are generated at a fixed rate and stored in a bucket up to a maximum capacity. A packet can be transmitted only when sufficient tokens are available. In a simple packet-based model, one token may permit the transmission of one packet.

Unlike the Leaky Bucket algorithm, unused tokens can accumulate, allowing several packets to be transmitted together when required.

Example: Suppose the token bucket has a capacity of five tokens and generates two tokens per second. If five tokens have accumulated, five packets can be transmitted immediately as a burst. Once the tokens are consumed, further transmission must wait for new tokens.

This makes Token Bucket useful for applications that generate occasional bursts of traffic while still requiring control over their long-term transmission rate.

3.5 Difference Between Leaky Bucket and Token Bucket

Basis	Leaky Bucket	Token Bucket
Working principle	Packets are transmitted at a fixed rate	Tokens permit packet transmission
Burst traffic	Smooths incoming bursts	Allows controlled bursts
Bucket stores	Packets or data	Tokens
Unused capacity	Does not accumulate transmission credits	Unused tokens accumulate

Excess traffic	Packets may be discarded if the queue fills	Packets must wait or may be dropped if tokens are insufficient
Main purpose	Produces smooth traffic	Controls average rate while allowing bursts

The main difference is that Leaky Bucket attempts to maintain a constant output rate, whereas Token Bucket allows temporary bursts when enough tokens are available.

3.6 Fragmentation

Fragmentation is the process of dividing a large IP packet into smaller fragments so that it can be transmitted over a network link with a smaller Maximum Transmission Unit (MTU).

The MTU is the maximum size of a packet that can be carried over a particular network link. Different networks may support different MTU values, so a packet that can travel through one network may be too large for another.

For example, consider an IPv4 packet of 4000 bytes that must pass through a network with an MTU of 1500 bytes. The original packet cannot be transmitted as a single unit over that link.

Assuming a 20-byte IPv4 header and no IP options, the packet contains 3980 bytes of data. It can be divided into three fragments with total lengths of 1500, 1500 and 1040 bytes, respectively. Each fragment has its own IP header.

Original IPv4 packet: 4000 bytes

4000-byte IP packet

Fragmentation for an MTU of 1500 bytes

Fragment 1: 1500 bytes

Fragment 2: 1500 bytes

Fragment 3: 1040 bytes

IPv4 uses the Identification, Flags and Fragment Offset fields in the IP header to support fragmentation and reassembly. The Identification field helps associate fragments with the original packet, while the Fragment Offset indicates their positions.

The Don't Fragment (DF) flag prevents fragmentation when set, and the More Fragments (MF) flag indicates whether additional fragments follow.

In IPv4, fragmentation may be performed by a router when permitted, and reassembly is performed at the destination. If fragmentation is prohibited and the packet exceeds the next link's MTU, the router drops the packet and normally sends an ICMP error message.

Although fragmentation allows packets to cross networks with smaller MTUs, it also introduces processing overhead. For this reason, modern networks often use Path MTU Discovery to avoid unnecessary fragmentation.

Conclusion

IP addressing, routing, congestion control and fragmentation are important concepts that support communication across computer networks. IP addresses identify devices and networks, while subnetting and supernetting help organize and manage address space efficiently.

Distance Vector and Link State routing algorithms determine least-cost paths using different methods of collecting and processing network information. Congestion-control mechanisms help regulate traffic and prevent network performance from deteriorating. Finally, fragmentation enables IPv4 packets to travel across links with different MTU requirements.

Together, these mechanisms help ensure that data can be addressed, forwarded and delivered efficiently across interconnected networks.